Servicio Nacional de Aprendizaje (SENA)
Análisis y Desarrollo de Software (ADSO) · Ficha 3145556

PROYECTO TU EVENTO

Documento de Patrones de Diseño

Catálogo de patrones arquitectónicos, de comportamiento, estructurales, de integración y de despliegue verificados en el código fuente

Autores	Jhampier Santos Ortiz Cristofer David Lozano Contreras
Programa de formación	Análisis y Desarrollo de Software (ADSO) Ficha 3145556
Instructora	Karol Daniela Correa
Fecha	Octubre de 2026
Versión	1.0

1. Introducción

Este documento cataloga los patrones de diseño utilizados en TuEvento, una plataforma para la gestión y venta de entradas de eventos compuesta por un backend en Spring Boot, un frontend web en React, una aplicación móvil en React Native y dos servicios auxiliares. Complementa los diagramas de arquitectura, de paquetes y de clases del producto de diseño del software.

Cada patrón se presenta con una estructura uniforme: categoría y capa, problema concreto que resuelve en TuEvento, ubicación exacta en el código, explicación de su implementación, fragmento de código real y beneficio aportado.

1.1 Criterio de inclusión

Solo se documentan patrones cuya implementación fue verificada en el código fuente del repositorio. Los fragmentos mostrados son extractos reales, abreviados cuando es necesario. Los patrones que aparecen en otras fuentes pero que no tienen una implementación deliberada en el código (por ejemplo CQRS, Template Method o una mensajería con broker externo) no se incluyen como patrones aplicados.

1.2 Convención de identificadores

Los patrones se identifican como P-01 a P-27 y se agrupan por afinidad en seis secciones. Las referencias cruzadas entre patrones usan ese identificador.

2. Resumen arquitectónico

TuEvento es un **monolito modular** con 14 módulos de dominio (security, profile, event, section, seat, ticket, payment, wallet, notification, storage, geolocation, category, language y theme) organizados bajo **Domain-Driven Design (DDD)** y **Arquitectura Hexagonal (puertos y adaptadores)**. Cada módulo tiene cuatro capas internas:

- **domain:** modelos, interfaces de repositorio y eventos de dominio.
- **application:** casos de uso, puertos de entrada y salida, DTOs, mappers y servicios de aplicación.
- **infrastructure:** persistencia JPA, adaptadores externos, canales de notificación y WebSocket.
- **interfaces/rest:** controllers.

Los tres clientes (frontend web, aplicación móvil y backend) se comunican exclusivamente por HTTP REST; las actualizaciones en tiempo real usan STOMP sobre SockJS/WebSocket. Los servicios auxiliares **fake-payment-gateway** y **nsfw-service** son aplicaciones independientes que el backend consume mediante adaptadores propios.

3. Tecnologías principales

Proyecto	Tecnología	Versión
Backend	Java	17
Backend	Spring Boot	3.5.12
Backend	PostgreSQL	17
Backend	Liquibase	Gestionada por Spring Boot
Backend	MapStruct	1.6.3
Backend	Lombok	Gestionada por Spring Boot
Backend	jjwt	0.12.6
Backend	AWS SDK (S3/MinIO)	2.25.60

Proyecto	Tecnología	Versión
Frontend web	React	18.2.0
Frontend web	Vite	5.0.0
Frontend web	@stomp/stompjs	7.3.0
App móvil	React Native	0.81.5
App móvil	Expo	~54.0.33
App móvil	@stomp/stompjs	7.3.0
fake-payment-gateway	Spring Boot	Mismo stack del backend
nsfw-service	Python / FastAPI + opennsfw2	No especificada
CI/CD	Jenkins + Docker + Kubernetes	No especificada

4. Diagramas de referencia

Los siguientes diagramas del repositorio de documentación complementan este catálogo:

Diagrama	Ruta
Arquitectura del sistema	`05_Diseno_Software/Diagrama de arquitectura.png`
Paquetes (estructura de módulos)	`05_Diseno_Software/Diagrama de paquetes.png`
Entidad-relación (DER completo)	`05_Diseno_Software/DER_TuEvento.png`
Procesos	`05_Diseno_Software/Diagrama de procesos.pdf`
Flujo	`05_Diseno_Software/diagramas de flujo .pdf`
Actividades por módulo (12 PNG)	`05_Diseno_Software/Diagrama de actividades (en modulos)/`
Secuencia	`03_Analisis_Software/secuencie Diagrams.pdf`
Clases: modelos, repositorios, servicios, DTOs, controllers, config y JWT	`03_Analisis_Software/class diagram/`
Casos de uso por módulo (16 PNG)	`03_Analisis_Software/Use case/`
MER por módulo (14 archivos)	`05_Diseno_Software/MER (en modulos)/`
Scripts SQL por módulo (13 archivos)	`05_Diseno_Software/SQL/`

5. Catálogo de patrones

5.1 Patrones arquitectónicos

Definen la estructura global del sistema: cómo se separan las responsabilidades, cómo se aísla el dominio y cómo se organizan los módulos.

P-01 | Arquitectura Hexagonal (Puertos y Adaptadores)

Categoría: Arquitectónico Capa: Backend

Problema que resuelve

La lógica de negocio (gestión de eventos, pagos, tickets) no debe depender de Spring, JPA ni de ningún proveedor externo. Así los casos de uso se pueden probar con mocks sin levantar base de datos ni capa REST.

Dónde está implementado

- Puerto de entrada: modules/security/application/port/in/LoginPort.java
- Puerto de salida: modules/security/application/port/out/TokenGeneratorPort.java
- Caso de uso: modules/security/application/usecase/LoginUseCase.java

Cómo se implementa

Cada módulo expone interfaces en application/port/in (lo que el exterior puede pedirle) y application/port/out (lo que el módulo necesita del exterior). Los controllers de interfaces/rest inyectan solo el puerto de entrada; los adaptadores de infrastructure implementan los puertos de salida.

Fragmento de código

application/port/in/LoginPort.java y application/usecase/LoginUseCase.java

```
public interface LoginPort {
    LoginResponse login(LoginRequest request);
}

@Service
@RequiredArgsConstructor
public class LoginUseCase implements LoginPort {
    private final PasswordEncoderPort passwordEncoder;  // puerto de salida
    private final TokenGeneratorPort  tokenGenerator;   // puerto de salida
    @Override
    @Transactional
    public LoginResponse login(LoginRequest request) { ... }
}
```

Beneficio

Los casos de uso se prueban inyectando mocks de PasswordEncoderPort y TokenGeneratorPort sin arrancar Spring. Cambiar la librería de tokens (jjwt por otra) no modifica el caso de uso.

P-02 | Domain-Driven Design: Aggregate Root, Value Object y Domain Event

Categoría: Arquitectónico Capa: Backend y fake-payment-gateway

Problema que resuelve

Los dominios de pagos y de eventos son complejos. Sin fronteras claras, cualquier servicio podría modificar el estado de un pago o de un evento directamente, rompiendo las reglas de negocio.

Dónde está implementado

- Aggregate root con fábrica estática: fake-payment-gateway/.../domain/model/Payment.java
- Value Object inmutable: fake-payment-gateway/.../domain/model/Money.java
- Modelo de dominio del evento: modules/event/domain/model/Event.java (con @Builder)
- Evento de dominio: modules/event/domain/event/EventStatusChangedEvent.java
- Repositorio de dominio (interfaz): modules/event/domain/repository/EventRepository.java

Cómo se implementa

Payment es el único punto de entrada para cambiar el estado de un pago: su constructor es privado, obliga a usar Payment.create(...) y cada método (approve(), refund(), etc.) valida la transición antes de mutar el estado. Money es final, sin setters y con validaciones en el constructor. EventRepository es una interfaz del dominio; su implementación JPA vive en infrastructure.

Fragmento de código

fake-payment-gateway/.../domain/model/Payment.java

```
public static Payment create(String externalReference,
        Money money, PaymentMethod paymentMethod) {
    Payment p = new Payment();
    p.paymentId = "fake_" + UUID.randomUUID();
    p.status    = PaymentStatus.PENDING;
    p.money     = money;
    p.createdAt = Instant.now();
    return p;
}
public PaymentStatusChanged approve() {
    transitionTo(PaymentStatus.APPROVED);
    return createStatusChangedEvent();
}
private void transitionTo(PaymentStatus newStatus) {
    if (!status.canTransitionTo(newStatus))
        throw new InvalidStatusTransitionException(...);
    this.status = newStatus;
}
```

Beneficio

Ningún servicio puede dejar un pago en APPROVED sin pasar por la máquina de estados. Toda la lógica de negocio queda en el dominio y no en los servicios.

P-03 | Monolito Modular

Categoría: Arquitectónico Capa: Backend

Problema que resuelve

El proyecto tiene un alcance amplio (14 módulos) y un equipo reducido. Se necesitan fronteras claras entre dominios para evitar el acoplamiento, pero sin el costo operativo de una arquitectura de microservicios.

Dónde está implementado

- Estructura de paquetes: modules/ con security, event, payment, wallet, notification, ticket, section, seat, storage, profile, geolocation, category, language y theme
- Comunicación entre módulos: EventStatusChangedListener.java consume el EventStatusChangedEvent publicado por ChangeEventStatusService

Cómo se implementa

Cada módulo tiene sus propias capas domain, application e infrastructure. La comunicación entre módulos se hace con eventos de dominio de Spring (ApplicationEventPublisher) en lugar de llamadas directas entre servicios, lo que mantiene el acoplamiento bajo.

Fragmento de código

modules/event/application/usecase/ChangeEventStatusService.java

```
eventPublisher.publishEvent(EventStatusChangedEvent.builder()
    .eventId(eventId)
    .newStatus(request.getNewStatus().name())
    .organizerId(event.getUserId())
    .eventName(event.getEventName())
    .reason(null)
    .build());
```

Beneficio

El módulo notification no importa nada del módulo event; solo reacciona al evento publicado. Si en el futuro se extrae notification como microservicio, el contrato sigue siendo el evento y no una dependencia directa.

5.2 Patrones de comportamiento y creacionales de lógica de negocio

Resuelven cómo colaboran los objetos y cómo se asignan responsabilidades en la lógica de negocio: control de estados, reacción a eventos, selección de algoritmos y cadenas de validación.

P-04 | State: ciclo de vida del evento

Categoría: Comportamiento Capa: Backend

Problema que resuelve

Un evento atraviesa varios estados (DRAFT, PENDING_REVIEW, PUBLISHED, REJECTED, CANCELLED y COMPLETED). Sin un control explícito de transiciones, un organizador podría, por ejemplo, pasar un evento de CANCELLED a PUBLISHED directamente.

Dónde está implementado

- Enum de estados: modules/event/domain/model/EventStatus.java
- Servicio que aplica la tabla de transiciones: modules/event/application/usecase/ChangeEventStatusService.java (método validateTransition)

Cómo se implementa

El switch de validateTransition actúa como tabla de transiciones explícita: cada combinación (estado actual, estado destino) permitida está declarada y cualquier otra lanza BusinessException con el código EVENT_INVALID_STATUS_TRANSITION.

Fragmento de código

modules/event/application/usecase/ChangeEventStatusService.java

```
private void validateTransition(EventStatus current, EventStatus next) {
    boolean allowed = switch (current) {
        case DRAFT          -> next == EventStatus.PENDING_REVIEW;
        case PENDING_REVIEW -> next == EventStatus.DRAFT;
        case REJECTED       -> next == EventStatus.DRAFT;
        case PUBLISHED      -> next == EventStatus.CANCELLED
                            || next == EventStatus.COMPLETED;
        default             -> false;
    };
    if (!allowed) throw new BusinessException(
        "EVENT_INVALID_STATUS_TRANSITION",
        "Transition from " + current + " to " + next + " is not allowed");
}
```

Beneficio

Agregar un estado o modificar una transición es un cambio localizado en un único método.

P-05 | State: máquina de estados del pago

Categoría: Comportamiento Capa: fake-payment-gateway

Problema que resuelve

El gateway de pago simula transiciones reales (PENDING, PROCESSING, APPROVED, REFUNDED). Sin este control, el sistema podría intentar reembolsar un pago que nunca fue aprobado.

Dónde está implementado

- fake-payment-gateway/.../domain/model/PaymentStatus.java

Cómo se implementa

PaymentStatus es un enum con el método abstracto allowedTransitions(). Cada constante lo implementa declarando los estados destino válidos, y canTransitionTo(newStatus) consulta ese conjunto. Si la transición no está permitida, Payment.transitionTo() lanza InvalidStatusTransitionException.

Fragmento de código

domain/model/PaymentStatus.java

```
public enum PaymentStatus {
    PENDING {
        public Set<PaymentStatus> allowedTransitions() {
            return Set.of(PROCESSING, CANCELLED);
        }
    },
    APPROVED {
        public Set<PaymentStatus> allowedTransitions() {
            return Set.of(REFUNDED);
        }
    },
    REFUNDED {
        public Set<PaymentStatus> allowedTransitions() {
            return Set.of(); // terminal
        }
    };
    public abstract Set<PaymentStatus> allowedTransitions();
    public boolean canTransitionTo(PaymentStatus next) {
        return allowedTransitions().contains(next);
    }
}
```

Beneficio

El comportamiento de cada estado queda encapsulado en el propio estado. Añadir un estado nuevo (por ejemplo DISPUTED) requiere solo una constante con sus transiciones.

P-06 | Observer / Event-Driven: cambio de estado del evento

Categoría: Comportamiento Capa: Backend

Problema que resuelve

Cuando un administrador aprueba o rechaza un evento, el módulo de notificaciones debe avisar al organizador. Pero ChangeEventStatusService no debe conocer ni depender del módulo de notificaciones.

Dónde está implementado

- Publicador: modules/event/application/usecase/ChangeEventStatusService.java (eventPublisher.publishEvent)
- Evento: modules/event/domain/event/EventStatusChangedEvent.java
- Suscriptor: modules/notification/infrastructure/messaging/EventStatusChangedListener.java

Cómo se implementa

El ApplicationEventPublisher de Spring actúa como bus de eventos. El listener usa @TransactionalEventListener(AFTER_COMMIT) para disparar la notificación solo si la transacción del cambio de estado se confirmó, y @Async para no bloquear la transacción original.

Fragmento de código

notification/infrastructure/messaging/EventStatusChangedListener.java

```
@Async
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void onEventStatusChanged(EventStatusChangedEvent event) {
    if (!PENDING_REVIEW.equals(event.getOldStatus())) return;
    String typeName = PUBLISHED.equals(event.getNewStatus())
            ? NotificationTypeNames.EVENT_PUBLISHED
            : NotificationTypeNames.EVENT_REJECTED;
    sendNotificationUseCase.execute(
        SendNotificationCommand.builder()
            .typeName(typeName)
            .userIds(List.of(event.getOrganizerId().intValue()))
            .eventName(event.getEventName())
            .build());
}
```

Beneficio

El módulo event nunca importa el módulo notification. Se pueden agregar nuevos suscriptores (por ejemplo, un webhook externo) sin modificar ChangeEventStatusService.

P-07 | Observer / Event-Driven: bloqueo de usuario (UserLockedEvent)

Categoría: Comportamiento Capa: Backend

Problema que resuelve

Cuando una cuenta se bloquea por intentos fallidos de inicio de sesión, otros módulos (notificaciones, auditoría) deben enterarse sin acoplar el módulo de seguridad a ellos.

Dónde está implementado

- Publicador: modules/security/application/usecase/LoginUseCase.java (eventPublisher.publishEvent)
- Evento: modules/security/domain/event/UserLockedEvent.java

Cómo se implementa

LoginUseCase publica un UserLockedEvent con el identificador del usuario, su alias y la fecha hasta la que queda bloqueado. Los módulos interesados lo consumen con la misma mecánica descrita en P-06.

Fragmento de código

security/application/usecase/LoginUseCase.java

```
eventPublisher.publishEvent(UserLockedEvent.builder()
    .userId(user.getUserId())
    .alias(user.getAlias())
    .lockedUntil(lockedUntil)
    .occurredAt(now)
    .build());
```

Beneficio

Separación limpia entre el módulo de seguridad y los módulos que reaccionan al bloqueo.

P-08 | Strategy: resolvers de OAuth por proveedor

Categoría: Comportamiento Capa: Backend

Problema que resuelve

El sistema permite iniciar sesión con Google y Facebook. El intercambio del código y la obtención del perfil son distintos en cada proveedor, pero el caso de uso no debe contener cadenas if (google) ... else if (facebook).

Dónde está implementado

- Registro de estrategias: shared/infrastructure/config/OauthConfig.java (bean oauthProviderResolvers)
- Consumidor: modules/security/application/usecase/OauthLoginUseCase.java (campo providerResolvers)

Cómo se implementa

Cada proveedor es una Function<String, OauthProfile> registrada en un Map con la clave "google" o "facebook". Al llegar una solicitud, OauthLoginUseCase busca la función por clave y la invoca. Para agregar otro proveedor basta con registrar una nueva entrada en el mapa.

Fragmento de código

shared/infrastructure/config/OauthConfig.java

```
@Bean
public Map<String, Function<String, OauthProfile>> oauthProviderResolvers() {
    return Map.of(
        "google",   googleProfileResolver(),
        "facebook", facebookProfileResolver()
    );
}
```

modules/security/application/usecase/OauthLoginUseCase.java

```
private final Map<String, Function<String, OauthProfile>> providerResolvers;
// Uso:
OauthProfile profile = providerResolvers
    .get(provider)
    .apply(authorizationCode);
```

Beneficio

Agregar un proveedor nuevo (GitHub, Apple) requiere solo una entrada en el mapa y una función en OauthConfig. El caso de uso no cambia.

P-09 | Strategy: canales de notificación

Categoría: Comportamiento Capa: Backend

Problema que resuelve

Las notificaciones deben entregarse por distintos canales (IN_APP por WebSocket y EMAIL por SMTP). La lógica de cada canal no debe mezclarse, y agregar un canal nuevo (por ejemplo SMS) no debe tocar el caso de uso.

Dónde está implementado

- Interfaz estrategia: modules/notification/application/port/out/NotificationChannelPort.java
- Estrategia IN_APP: modules/notification/infrastructure/channel/InAppChannelAdapter.java
- Estrategia EMAIL: modules/notification/infrastructure/channel/EmailChannelAdapter.java
- Selector de estrategia: modules/notification/application/usecase/SendNotificationUseCase.java

Cómo se implementa

El constructor de SendNotificationUseCase inyecta la lista de NotificationChannelPort y la convierte en un mapa usando channelName() como clave. Para cada notificación selecciona del mapa el adaptador correspondiente.

Fragmento de código

application/port/out/NotificationChannelPort.java y SendNotificationUseCase.java

```
public interface NotificationChannelPort {
    String channelName();
    void deliver(Notification notification, List<Integer> userIds);
}

// SendNotificationUseCase (constructor)
this.channelsByName = channelPorts.stream()
    .collect(Collectors.toMap(
        NotificationChannelPort::channelName,
        Function.identity()));
```

Beneficio

InAppChannelAdapter y EmailChannelAdapter son independientes. Agregar un SmsChannelAdapter requiere implementar la interfaz y registrar el bean, sin modificar el caso de uso.

P-10 | Factory Method: NotificationMessageFactory

Categoría: Creacional Capa: Backend

Problema que resuelve

El texto de la notificación varía según el tipo (PAYMENT_APPROVED, EVENT_PUBLISHED, EVENT_REJECTED, etc.) y según el canal (HTML para correo, texto plano para IN_APP). Sin una fábrica central, esa lógica estaría dispersa en listeners y servicios.

Dónde está implementado

- modules/notification/application/service/NotificationMessageFactory.java

Cómo se implementa

Clase final con constructor privado y método estático build(typeName, channelName, command). Un switch sobre typeName selecciona el método privado de construcción correspondiente, y cada uno ramifica según el canal para producir HTML o texto plano.

Fragmento de código

notification/application/service/NotificationMessageFactory.java

```
public static MessageContent build(String typeName,
        String channelName, SendNotificationCommand command) {
    boolean email = NotificationChannelNames.EMAIL.equals(channelName);
    return switch (typeName) {
        case NotificationTypeNames.PAYMENT_APPROVED -> paymentApproved(email, command);
        case NotificationTypeNames.EVENT_PUBLISHED  -> eventPublished(email, command);
        case NotificationTypeNames.EVENT_REJECTED   -> eventRejected(email, command);
        default -> new MessageContent("Notificación",
            email ? html("Tienes una nueva notificación.")
                  : "Tienes una nueva notificación.");
    };
}
```

Beneficio

Agregar un tipo de notificación es un case nuevo y un método privado. Además, la mitigación de XSS sobre el motivo de rechazo está centralizada en escapeHtml() dentro de la misma clase.

P-11 | Chain of Responsibility: moderación de imágenes en cascada

Categoría: Comportamiento Capa: Backend

Problema que resuelve

Las imágenes de eventos se filtran primero por contenido sexual (servicio local OpenNSFW2) y luego, solo si pasan el primer filtro, por contenido violento (API Sightengine). Así solo se llama al servicio de pago cuando es necesario, reduciendo costos.

Dónde está implementado

- Orquestador de la cadena: modules/storage/application/service/ImageModerationService.java
- Eslabón 1: modules/storage/infrastructure/external/NsfwClientAdapter.java
- Eslabón 2: modules/storage/infrastructure/external/SightengineAdapter.java
- Invocador: modules/storage/application/usecase/UploadFileUseCase.java (moderationService.moderate)

Cómo se implementa

ImageModerationService.moderate() llama primero a nsfwClient.isSafe(). Si retorna false, lanza ImagePolicyViolationException("NSFW_CONTENT") sin continuar. Solo si retorna true consulta a sightengineAdapter.isSafe(). Ambos adaptadores aplican una política fail-open: si el servicio externo falla, devuelven true para no bloquear la subida.

Fragmento de código

storage/application/service/ImageModerationService.java

```
public void moderate(byte[] imageBytes) {
    if (!isNsfwSafe(imageBytes)) {
        log.warn("Image rejected by NSFW filter");
        throw new ImagePolicyViolationException("NSFW_CONTENT");
    }
    if (!isGoreSafe(imageBytes)) {
        log.warn("Image rejected by gore filter");
        throw new ImagePolicyViolationException("VIOLENT_CONTENT");
    }
}
```

Beneficio

Agregar un tercer filtro (por ejemplo, detección de logos) requiere implementar ModerationPort y añadir un paso en moderate(), sin tocar los otros eslabones.

5.3 Patrones estructurales y de creación de objetos

Organizan la composición de clases y la separación entre capas: adaptación de servicios externos, acceso a datos, conversión entre modelos y construcción de objetos.

P-12 | Adapter: pasarela de pago

Categoría: Estructural Capa: Backend

Problema que resuelve

El sistema de pagos debe poder trabajar con el gateway de pruebas local y, en producción, con otro proveedor (Wompi). El caso de uso de pago no puede acoplarse a la API HTTP específica de ninguno.

Dónde está implementado

- Puerto de salida: modules/payment/application/port/out/PaymentGatewayPort.java
- Adaptador concreto: modules/payment/infrastructure/external/FakePaymentGatewayAdapter.java
- Activación condicional: @ConditionalOnProperty(name = "payment.gateway", havingValue = "fake")

Cómo se implementa

PaymentGatewayPort define createPayment, getPayment, cancelPayment, refundPayment y processWebhook. FakePaymentGatewayAdapter los implementa traduciendo cada llamada a la API REST del fake-gateway con RestTemplate. El bean se activa solo si payment.gateway=fake, lo que permite registrar un adaptador de producción sin cambiar el caso de uso.

Fragmento de código

payment/application/port/out/PaymentGatewayPort.java y FakePaymentGatewayAdapter.java

```
public interface PaymentGatewayPort {
    GatewayPayment createPayment(CreatePaymentCommand command);
    GatewayPayment getPayment(String gatewayPaymentId);
    void cancelPayment(String gatewayPaymentId);
    void refundPayment(String gatewayPaymentId);
    GatewayPaymentEvent processWebhook(String payload, String signature);
}

@Component
@ConditionalOnProperty(name = "payment.gateway", havingValue = "fake")
public class FakePaymentGatewayAdapter implements PaymentGatewayPort {
    @Override
    public GatewayPayment createPayment(CreatePaymentCommand command) {
        ResponseEntity<JsonNode> response = restTemplate.exchange(
            gatewayUrl + "/public/payments", HttpMethod.POST, entity, JsonNode.class);
        return GatewayPayment.builder()
            .paymentId(body.get("paymentId").asText())
            .status(body.get("status").asText())
            .build();
    }
}
```

Beneficio

Cambiar a otro proveedor en producción consiste en registrar un nuevo adaptador y cambiar una propiedad de configuración. El caso de uso de pago no sabe a qué gateway apunta.

P-13 | Adapter: moderación de imágenes (NSFW y Sightengine)

Categoría: Estructural Capa: Backend

Problema que resuelve

El backend Java debe consumir dos servicios externos heterogéneos: el microservicio Python nsfw-service (REST multipart) y la API comercial Sightengine. La lógica de moderación no debe conocer detalles HTTP de ninguno.

Dónde está implementado

- Interfaz: modules/storage/application/port/out/ModerationPort.java
- Adaptador NSFW: modules/storage/infrastructure/external/NsfwClientAdapter.java
- Adaptador Sightengine: modules/storage/infrastructure/external/SightengineAdapter.java

Cómo se implementa

NsfwClientAdapter.isSafe() llama a POST {nsfw_url}/classify enviando la imagen como multipart/form-data, interpreta el campo nsfw_score de la respuesta JSON y lo compara con el umbral configurado. Cualquier excepción (tiempo de espera, servicio caído) devuelve true (fail-open).

Fragmento de código

storage/infrastructure/external/NsfwClientAdapter.java

```
public boolean isSafe(byte[] imageBytes) {
    try {
        String url = moderationConfig.getNsfw().getUrl() + "/classify";
        // construye multipart form...
        ResponseEntity<Map> response = rt.postForEntity(url, entity, Map.class);
        double score = ((Number) response.getBody().get("nsfw_score")).doubleValue();
        return score < moderationConfig.getNsfw().getThreshold();
    } catch (Exception e) {
        log.error("NSFW service unreachable - failing open");
        return true;  // fail-open
    }
}
```

Beneficio

Si se reemplaza el servicio NSFW local por otro (por ejemplo AWS Rekognition), solo cambia el adaptador y no la cadena de moderación.

P-14 | Repository: abstracción de persistencia con implementación JPA

Categoría: Estructural Capa: Backend

Problema que resuelve

El dominio de eventos no debe depender de JPA, EntityManager ni de detalles de persistencia. Las consultas del dominio deben expresarse en términos del negocio y no de SQL.

Dónde está implementado

- Interfaz de dominio: modules/event/domain/repository/EventRepository.java
- Implementación JPA: modules/event/infrastructure/persistence/repository/EventRepositoryImpl.java
- Mapper entidad-dominio: modules/event/application/mapper/EventEntityMapper.java (MapStruct)

Cómo se implementa

EventRepository define métodos en lenguaje de negocio, como findAllPublishedExpiredBefore(LocalDate) o findByCityId(Long). EventRepositoryImpl los implementa inyectando EventJpaRepository (Spring Data JPA) y EventEntityMapper para convertir entre EventEntity (JPA) y Event (dominio).

Fragmento de código

infrastructure/persistence/repository/EventRepositoryImpl.java

```
@Repository
@RequiredArgsConstructor
public class EventRepositoryImpl implements EventRepository {
    private final EventJpaRepository jpaRepository;
    private final EventEntityMapper  mapper;

    @Override
    public Optional<Event> findById(Long eventId) {
        return jpaRepository.findById(eventId)
            .map(mapper::toDomain);  // entidad JPA -> objeto de dominio
    }
    @Override
    public Event save(Event event) {
        return mapper.toDomain(jpaRepository.save(mapper.toEntity(event)));
    }
}
```

Beneficio

Migrar de PostgreSQL a otro motor requiere solo reemplazar la implementación; los casos de uso no cambian.

P-15 | Mapper / DTO: MapStruct para separación de capas

Categoría: Estructural Capa: Backend

Problema que resuelve

El modelo de dominio Event y la entidad JPA EventEntity tienen campos diferenciados (auditoría, relaciones lazy, etc.) que no deben exponerse ni mezclarse. Copiar propiedades a mano es propenso a errores.

Dónde está implementado

- modules/event/application/mapper/EventEntityMapper.java

Cómo se implementa

MapStruct genera en tiempo de compilación la implementación del mapper a partir de una interfaz anotada con @Mapper(componentModel = "spring"). Las anotaciones @Mapping(target = "X", ignore = true) excluyen los campos de auditoría al construir el dominio desde la entidad.

Fragmento de código

event/application/mapper/EventEntityMapper.java

```
@Mapper(componentModel = "spring")
public interface EventEntityMapper {
    @Mapping(target = "createdAt", ignore = true)
    @Mapping(target = "updatedAt", ignore = true)
    @Mapping(target = "createdBy",  ignore = true)
    @Mapping(target = "updatedBy",  ignore = true)
    Event toDomain(EventEntity entity);

    EventEntity toEntity(Event domain);
}
```

Beneficio

Elimina el código repetitivo de conversión. Si se agrega un campo a Event, MapStruct detecta en compilación que falta el mapeo, evitando errores silenciosos.

P-16 | Builder: Lombok @Builder en modelos, eventos y DTOs

Categoría: Creacional Capa: Backend

Problema que resuelve

Objetos como AuthSession, RefreshToken, LoginResponse y EventStatusChangedEvent tienen muchos campos, varios opcionales. Los constructores con ocho o más parámetros son ilegibles y propensos a errores de orden.

Dónde está implementado

- modules/security/domain/event/UserLockedEvent.java (@Builder)
- modules/event/domain/event/EventStatusChangedEvent.java (@Builder)
- modules/event/domain/model/Event.java (@Builder)
- Uso en LoginUseCase.java para construir AuthSession, RefreshToken y LoginResponse

Cómo se implementa

Lombok genera el builder en tiempo de compilación; los casos de uso construyen los objetos encadenando métodos con nombre de campo.

Fragmento de código

security/application/usecase/LoginUseCase.java

```
AuthSession session = authSessionRepository.save(
    AuthSession.builder()
        .user(user)
        .accessToken(accessToken)
        .issuedAt(now)
        .expiresAt(now.plusMinutes(ACCESS_TOKEN_MINUTES))
        .revoked(false)
        .build());
return LoginResponse.builder()
    .accessToken(accessToken)
    .refreshToken(refreshToken)
    .userId(user.getUserId())
    .role(user.getRole().getCode())
    .build();
```

Beneficio

Mayor legibilidad y seguridad en compilación. Agregar un campo opcional no rompe el código existente.

P-17 | Value Object: Money

Categoría: DDD / Estructural Capa: fake-payment-gateway

Problema que resuelve

El monto de un pago y su moneda deben ser siempre coherentes. Un BigDecimal suelto puede ser negativo o carecer de moneda asociada.

Dónde está implementado

- fake-payment-gateway/.../domain/model/Money.java

Cómo se implementa

Money es una clase final e inmutable: valida en el constructor que el monto no sea nulo ni negativo y que la moneda no esté vacía, normaliza la moneda a mayúsculas y no expone setters.

Fragmento de código

domain/model/Money.java

```
public final class Money {
    private final BigDecimal amount;
    private final String currency;
    public Money(BigDecimal amount, String currency) {
        if (amount == null || amount.compareTo(BigDecimal.ZERO) < 0)
            throw new IllegalArgumentException("Amount must be non-negative");
        if (currency == null || currency.isBlank())
            throw new IllegalArgumentException("Currency cannot be null");
        this.amount   = amount;
        this.currency = currency.toUpperCase();
    }
    // sin setters: inmutable
}
```

Beneficio

Es imposible crear un Money con monto negativo o moneda vacía. Al ser inmutable, es seguro en contextos concurrentes y dentro de eventos de dominio.

5.4 Patrones de integración y despliegue

Cubren la comunicación con servicios externos, la tolerancia a fallos y la automatización del despliegue sobre contenedores y Kubernetes.

P-18 | Adapter con reintentos y firma HMAC: WebhookNotifierAdapter

Categoría: Integración Capa: fake-payment-gateway

Problema que resuelve

El fake-gateway debe notificar al backend principal cada vez que un pago cambia de estado. La red puede fallar de forma transitoria y el receptor debe verificar que el webhook es auténtico y no fue manipulado.

Dónde está implementado

- Interfaz: fake-payment-gateway/.../application/port/out/WebhookNotifierPort.java
- Adaptador: fake-payment-gateway/.../infrastructure/webhook/WebhookNotifierAdapter.java

Cómo se implementa

El primer intento es síncrono y participa en la transacción. Si falla, retryAsync() reintenta hasta 3 veces con espera creciente (2 s, 10 s y 30 s). Cada intento firma el payload con HMAC-SHA256 usando una clave compartida; el backend (FakePaymentGatewayAdapter) verifica la misma firma antes de procesar el evento.

Fragmento de código

infrastructure/webhook/WebhookNotifierAdapter.java

```
private static final int    MAX_ATTEMPTS       = 3;
private static final long[] BACKOFF_DELAYS_MS = {2000, 10000, 30000};

@Override
public void notify(PaymentStatusChanged event) {
    boolean success = attemptDelivery(event, 1);
    if (!success) retryAsync(event, 2); // reintentos asíncronos
}
@Async
private void retryAsync(PaymentStatusChanged event, int startAttempt) {
    for (int attempt = startAttempt; attempt <= MAX_ATTEMPTS; attempt++) {
        Thread.sleep(BACKOFF_DELAYS_MS[attempt - 2]);
        if (attemptDelivery(event, attempt)) return;
    }
}
```

Beneficio

Los cortes transitorios de red no pierden notificaciones de pago, y la firma HMAC impide que un atacante inyecte webhooks falsos.

P-19 | Microservicio independiente: nsfw-service (Python / FastAPI)

Categoría: Arquitectónico / Integración Capa: nsfw-service

Problema que resuelve

Ejecutar inferencia de modelos de aprendizaje automático dentro de la JVM del backend Spring Boot es ineficiente y agrega dependencias pesadas. La clasificación NSFW se aísla en un proceso Python propio.

Dónde está implementado

- nsfw-service/main.py

Cómo se implementa

FastAPI expone POST /classify, que recibe una imagen multipart, la pasa a opennsfw2.predict_image() y devuelve nsfw_score y flagged. El backend Java lo consume mediante NsfwClientAdapter (ver P-13).

Fragmento de código

nsfw-service/main.py

```
@app.post("/classify")
async def classify(file: UploadFile = File(...)):
    raw = await file.read()
    nsfw_score = opennsfw2.predict_image(io.BytesIO(raw))
    return {
        "nsfw_score": round(float(nsfw_score), 4),
        "flagged":    nsfw_score >= 0.6,
    }
```

Beneficio

El modelo puede actualizarse sin recompilar ni redesplegar el backend Java, y el servicio puede escalar de forma independiente. El umbral utilizado por el backend es configurable mediante ModerationConfig.

P-20 | Pipeline CI/CD con etapas secuenciales (Jenkins)

Categoría: Despliegue / Integración Capa: Infraestructura

Problema que resuelve

Validar, empaquetar y publicar la aplicación de forma manual permite que un commit defectuoso llegue a producción sin pasar por compilación ni empaquetado.

Dónde está implementado

- Jenkinsfile (raíz del repositorio de código)

Cómo se implementa

El Jenkinsfile declara las etapas en orden: Checkout, Clean & Compile Backend, Unit Tests, Package Backend JAR, Build Backend Docker Image, Build Frontend, Build Frontend Docker Image, Push to Registry y Deploy to Kubernetes. Si una etapa falla, el pipeline se detiene; el bloque post notifica el resultado.

Fragmento de código

Jenkinsfile (fragmento)

```
pipeline {
    environment {
        BACKEND_IMAGE   = "tu-evento-backend:${env.BUILD_ID}"
        DOCKER_REGISTRY = "jhampier23/tu-evento"
    }
    stages {
        stage('Package Backend JAR') {
            steps {
                dir('TuEventoBackend/tu-evento') {
                    sh './mvnw clean package -DskipTests'
                }
            }
        }
        stage('Build Backend Docker Image') {
            steps {
                dir('TuEventoBackend/tu-evento') {
                    sh "docker build -t ${BACKEND_IMAGE} ."
                }
            }
        }
        stage('Push to Registry') {
            steps {
                sh "docker tag ${BACKEND_IMAGE} ${DOCKER_REGISTRY}:${env.BUILD_ID}"
                sh "docker push ${DOCKER_REGISTRY}:${env.BUILD_ID}"
            }
        }
    }
}
```

Beneficio

Cada build produce una imagen Docker etiquetada con el BUILD_ID, lo que garantiza trazabilidad entre el código y el artefacto desplegado.

P-21 | Horizontal Pod Autoscaler (HPA): escalado automático en Kubernetes

Categoría: Despliegue / Disponibilidad Capa: Infraestructura

Problema que resuelve

Durante la venta de entradas de un evento popular, el tráfico hacia el backend puede multiplicarse en segundos. Con un número fijo de réplicas el sistema se saturaría.

Dónde está implementado

- k8s/hpa.yaml

Cómo se implementa

El HPA monitorea CPU y memoria del Deployment tu-evento-backend: si el uso de CPU supera el 70 % o el de memoria el 80 %, escala hasta 10 réplicas. La política scaleDown tiene una ventana de estabilización de 5 minutos para evitar oscilaciones continuas.

Fragmento de código

k8s/hpa.yaml (fragmento)

```
spec:
  scaleTargetRef:
    name: tu-evento-backend
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
```

Beneficio

El sistema escala automáticamente ante picos de demanda, como la apertura de venta de entradas, sin intervención manual.

5.5 Patrones en el frontend web (React)

Gestión de estado global, reutilización de lógica con estado y control de concurrencia en las peticiones HTTP de la aplicación web.

P-22 | Context / Provider: ThemeContext y NotificationContext

Categoría: Comportamiento / Estructural Capa: Frontend web

Problema que resuelve

La paleta de colores del tema activo y las notificaciones en tiempo real deben estar disponibles en cualquier componente sin pasar props manualmente a través de toda la jerarquía.

Dónde está implementado

- TuEventoFrontend/web/src/context/ThemeContext.jsx (ThemeProvider y useTheme)
- TuEventoFrontend/web/src/context/NotificationContext.jsx (NotificationProvider y useNotifications)

Cómo se implementa

ThemeProvider mantiene la paleta activa en el estado de React, la aplica al DOM como propiedades CSS personalizadas (--color-primary, etc.) y expone applyTheme, resetTheme y syncTheme. NotificationProvider mantiene un único cliente WebSocket STOMP centralizado y un mapa de suscripciones activas, y expone subscribe(destination, callback, key) para que cualquier componente se suscriba dinámicamente.

Fragmento de código

web/src/context/ThemeContext.jsx

```
export const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [palette, setPalette] = useState(DEFAULT_PALETTE);

  // Aplica la paleta como CSS custom properties en :root
  const applyPaletteToDOM = (palette) => {
    Object.entries(palette).forEach(([key, value]) => {
      document.documentElement.style
        .setProperty(`--color-${key}`, value);
    });
  };
  return (
    <ThemeContext.Provider value={{ palette, applyTheme, resetTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}
export const useTheme = () => useContext(ThemeContext);
```

Beneficio

Cualquier componente accede al tema con useTheme() sin conocer cómo se carga o sincroniza. El WebSocket de NotificationProvider se inicializa una sola vez para toda la aplicación, evitando conexiones duplicadas.

P-23 | Custom Hook: useTokenRefresh y useToast

Categoría: Comportamiento / Reutilización Capa: Frontend web

Problema que resuelve

El refresco automático del JWT y el manejo de alertas toast son lógicas con estado que se necesitan en varios componentes. Sin hooks habría useEffect y useRef duplicados en cada página.

Dónde está implementado

- TuEventoFrontend/web/src/hooks/useTokenRefresh.js
- TuEventoFrontend/web/src/hooks/useToast.js

Cómo se implementa

useTokenRefresh(onTokenRefreshed) encapsula el intervalo de refresco (cada 12 minutos), evita refrescos simultáneos con isRefreshing.current y hace un cierre de sesión limpio si el refresco falla con 401. Recibe un callback que NotificationContext usa para reconectar el WebSocket con el nuevo token. useToast encapsula el estado del toast, el temporizador de ocultado automático (5 segundos) y su reinicio si se lanza la misma alerta antes de expirar.

Fragmento de código

web/src/hooks/useTokenRefresh.js

```
export const useTokenRefresh = (onTokenRefreshed) => {
  const isRefreshing = useRef(false);

  const refreshToken = useCallback(async () => {
    if (isRefreshing.current) return; // evita condición de carrera
    isRefreshing.current = true;
    try {
      const data = await httpRequest(`${API_URL}/auth/refresh`, {
        method: 'POST',
        body: JSON.stringify({ refreshToken: localStorage.getItem('refreshToken') }),
      });
      localStorage.setItem('token', data.accessToken);
      onTokenRefreshed?.(data.accessToken); // notifica al WebSocket
    } finally {
      isRefreshing.current = false;
    }
  }, [onTokenRefreshed]);

  useEffect(() => {
    const id = setInterval(refreshToken, REFRESH_INTERVAL_MS); // 12 min
    return () => clearInterval(id);
  }, [refreshToken]);
};
```

Beneficio

La lógica de refresco queda en un único lugar. Si cambia la URL del endpoint o el intervalo, se modifica el hook y toda la aplicación queda actualizada.

P-24 | Interceptor con promesa compartida (singleton de refresco) en httpClient

Categoría: Comportamiento / Integración Capa: Frontend web

Problema que resuelve

Si diez peticiones simultáneas reciben un 401 (token expirado), sin control harían diez llamadas paralelas a /auth/refresh, agotando los refresh tokens o provocando errores de concurrencia.

Dónde está implementado

- TuEventoFrontend/web/src/services/httpClient.js

Cómo se implementa

refreshPromise es una variable de módulo (singleton). La primera petición con 401 crea la promesa de refresco y la asigna a refreshPromise; las siguientes detectan que no es null y esperan la misma promesa. Al terminar, todas reintentan con el nuevo token.

Fragmento de código

web/src/services/httpClient.js

```
let refreshPromise = null; // singleton de módulo

async function refreshAccessToken() {
  if (refreshPromise) return refreshPromise; // se encola en la misma promesa
  refreshPromise = (async () => {
    const res = await fetch(`${API_URL}/auth/refresh`, { method: 'POST', ... });
    const { data } = await res.json();
    localStorage.setItem('token', data.accessToken);
    return data.accessToken;
  })();
  try {
    return await refreshPromise;
  } finally {
    refreshPromise = null; // libera para el siguiente ciclo
  }
}
```

Beneficio

Solo se hace una llamada a /auth/refresh aunque haya decenas de peticiones fallidas simultáneas, evitando agotar los refresh tokens.

5.6 Patrones en la aplicación móvil (React Native / Expo) y transversales

Gestión de sesión y tema en el dispositivo, optimización de memoria para mapas de asientos de gran aforo y capa de servicios compartida por web y móvil.

P-25 | Context / Provider: AuthContext y ThemeContext (móvil)

Categoría: Comportamiento / Estructural Capa: App móvil

Problema que resuelve

El estado de sesión del usuario y la paleta de colores deben estar disponibles en toda la aplicación sin pasar props a través de cada pantalla.

Dónde está implementado

- TuEventoFrontend/movil/src/context/AuthContext.jsx (AuthProvider y useAuth)
- TuEventoFrontend/movil/src/context/ThemeContext.jsx (ThemeProvider y useTheme)

Cómo se implementa

AuthProvider persiste el token en AsyncStorage, restaura la sesión al arrancar leyendo y decodificando el JWT con jwtDecode, y expone login(), logout() y setSession(). ThemeProvider se sincroniza con el backend mediante getActivePalette() y reacciona cuando cambia el usuario (al cerrar sesión restablece el tema PRINCIPAL).

Fragmento de código

movil/src/context/AuthContext.jsx

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    const restoreSession = async () => {
      const accessToken = await AsyncStorage.getItem("accessToken");
      if (accessToken) {
        const decoded = jwtDecode(accessToken);
        if (decoded.exp * 1000 < Date.now()) {
          await AsyncStorage.multiRemove(["accessToken", "refreshToken"]);
        } else {
          const profile = await profileService.getByUserId(decoded.userId, accessToken);
          setUser({ userId: decoded.userId, role: decoded.role, ...profile });
        }
      }
    };
    restoreSession();
  }, []);
  return (
    <AuthContext.Provider value={{ user, login, logout, setSession }}>
      {children}
    </AuthContext.Provider>
  );
}
```

Beneficio

La restauración de sesión al arrancar y el cierre de sesión quedan centralizados. Cualquier pantalla accede al usuario con useAuth() sin conocer AsyncStorage.

P-26 | Custom Hook con diseño orientado a datos: useSeatStore

Categoría: Comportamiento / Rendimiento Capa: App móvil

Problema que resuelve

El selector de asientos puede manejar hasta decenas de miles de sillas a la vez (venues de gran aforo). Un mapa de objetos de React normales provocaría re-renderizados masivos y agotaría la memoria del dispositivo.

Dónde está implementado

- TuEventoFrontend/movil/src/hooks/useSeatStore.js

Cómo se implementa

useSeatStore encapsula SeatDataStore, que usa arreglos tipados (TypedArrays) en lugar de objetos JavaScript para reducir el consumo de memoria. El hook expone loadSeats, updateSeat, getSeat y cart. El re-renderizado se dispara solo incrementando un contador version cuando algo cambia, en lugar de mutar el estado directamente.

Fragmento de código

movil/src/hooks/useSeatStore.js

```
export function useSeatStore(currentUserId) {
  const storeRef = useRef(null);
  const [version, setVersion] = useState(0);
  if (!storeRef.current) {
    storeRef.current = new SeatDataStore(150000); // hasta 150 000 sillas
  }
  const updateSeat = useCallback((seatId, updates) => {
    const changed = storeRef.current.updateSeat(seatId, updates, currentUserId);
    if (changed) setVersion(v => v + 1); // re-render solo si cambió
    return changed;
  }, [currentUserId]);
  const seats = useMemo(() =>
    storeRef.current.getAllSeatsAsObject(), [version]);
  return { seats, cart, loadSeats, updateSeat, getSeat };
}
```

Beneficio

Soporta venues de gran aforo en dispositivos móviles con memoria limitada sin degradación perceptible del rendimiento.

P-27 | Service Layer: capa de servicios API (web y móvil)

Categoría: Estructural Capa: Frontend web y app móvil

Problema que resuelve

Los componentes no deben hacer llamadas fetch directamente. Sin una capa de servicios, la URL del backend, los encabezados de autorización y el manejo de errores quedarían duplicados en decenas de componentes.

Dónde está implementado

- Web: TuEventoFrontend/web/src/services/ (EventService.js, NotificationService.js, themeService.js, etc.)
- Móvil: TuEventoFrontend/movil/src/services/ (authService.js, eventService.js, paymentService.js, seatService.js, websocketService.js, etc.)

Cómo se implementa

Cada archivo de servicio exporta funciones que encapsulan el endpoint, el método HTTP, los encabezados y el parseo de la respuesta. En la web, los servicios autenticados usan httpClient.httpRequest() (que gestiona el refresco automático, P-24). En el móvil, los servicios leen el token desde AsyncStorage antes de cada llamada.

Fragmento de código

movil/src/services/websocketService.js

```
export async function connectSeatSocket(eventId, onSeatUpdate) {
  const token = await AsyncStorage.getItem('accessToken');
  const client = new Client({
    webSocketFactory: () =>
      new SockJS(`${getBaseURL()}/ws?token=${token}`),
    onConnect: () => {
      client.subscribe(`/topic/events/${eventId}/seats`, (msg) => {
        onSeatUpdate(JSON.parse(msg.body));
      });
    },
    reconnectDelay: 5000,
  });
  client.activate();
  return client; // el componente lo desactiva en el cleanup
}
```

Beneficio

Los componentes son agnósticos a la URL del backend. Cambiar el baseURL o la estrategia de autenticación requiere modificar solo la capa de servicios.

6. Tabla resumen

Patrón	Categoría	Capa	Archivo principal
P-01 Arquitectura Hexagonal (Puertos y Adaptadores)	Arquitectónico	Backend	`modules/security/application/port/in/LoginPort.java`
P-02 Domain-Driven Design: Aggregate Root, Value Object y Domain Event	Arquitectónico	Backend y fake-payment-gateway	`fake-payment-gateway/.../domain/model/Payment.java`
P-03 Monolito Modular	Arquitectónico	Backend	`modules/ con security, event, payment, wallet, notification, ticket, section, seat, storage, profile, geolocation, category, language y theme`
P-04 State: ciclo de vida del evento	Comportamiento	Backend	`modules/event/domain/model/EventStatus.java`
P-05 State: máquina de estados del pago	Comportamiento	fake-payment-gateway	`fake-payment-gateway/.../domain/model/PaymentStatus.java`
P-06 Observer / Event-Driven: cambio de estado del evento	Comportamiento	Backend	`modules/event/application/usecase/ChangeEventStatusService.java`
P-07 Observer / Event-Driven: bloqueo de usuario (UserLockedEvent)	Comportamiento	Backend	`modules/security/application/usecase/LoginUseCase.java`
P-08 Strategy: resolvers de OAuth por proveedor	Comportamiento	Backend	`shared/infrastructure/config/OauthConfig.java`
P-09 Strategy: canales de notificación	Comportamiento	Backend	`modules/notification/application/port/out/NotificationChannelPort.java`
P-10 Factory Method: NotificationMessageFactory	Creacional	Backend	`modules/notification/application/service/NotificationMessageFactory.java`
P-11 Chain of Responsibility: moderación de imágenes en cascada	Comportamiento	Backend	`modules/storage/application/service/ImageModerationService.java`
P-12 Adapter: pasarela de pago	Estructural	Backend	`modules/payment/application/port/out/PaymentGatewayPort.java`
P-13 Adapter: moderación de imágenes (NSFW y Sightengine)	Estructural	Backend	`modules/storage/application/port/out/ModerationPort.java`
P-14 Repository: abstracción de persistencia con implementación JPA	Estructural	Backend	`modules/event/domain/repository/EventRepository.java`
P-15 Mapper / DTO: MapStruct para separación de capas	Estructural	Backend	`modules/event/application/mapper/EventEntityMapper.java`
P-16 Builder: Lombok @Builder en modelos, eventos y DTOs	Creacional	Backend	`modules/security/domain/event/UserLockedEvent.java`
P-17 Value Object: Money	DDD / Estructural	fake-payment-gateway	`fake-payment-gateway/.../domain/model/Money.java`
P-18 Adapter con reintentos y firma HMAC: WebhookNotifierAdapter	Integración	fake-payment-gateway	`fake-payment-gateway/.../application/port/out/WebhookNotifierPort.java`
P-19 Microservicio independiente: nsfw-service (Python / FastAPI)	Arquitectónico / Integración	nsfw-service	`nsfw-service/main.py`
P-20 Pipeline CI/CD con etapas secuenciales (Jenkins)	Despliegue / Integración	Infraestructura	`Jenkinsfile`
P-21 Horizontal Pod Autoscaler (HPA): escalado automático en Kubernetes	Despliegue / Disponibilidad	Infraestructura	`k8s/hpa.yaml`
P-22 Context / Provider: ThemeContext y NotificationContext	Comportamiento / Estructural	Frontend web	`TuEventoFrontend/web/src/context/ThemeContext.jsx`

Patrón	Categoría	Capa	Archivo principal
P-23 Custom Hook: useTokenRefresh y useToast	Comportamiento / Reutilización	Frontend web	`TuEventoFrontend/web/src/hooks/useTokenRefresh.js`
P-24 Interceptor con promesa compartida (singleton de refresco) en httpClient	Comportamiento / Integración	Frontend web	`TuEventoFrontend/web/src/services/httpClient.js`
P-25 Context / Provider: AuthContext y ThemeContext (móvil)	Comportamiento / Estructural	App móvil	`TuEventoFrontend/movil/src/context/AuthContext.jsx`
P-26 Custom Hook con diseño orientado a datos: useSeatStore	Comportamiento / Rendimiento	App móvil	`TuEventoFrontend/movil/src/hooks/useSeatStore.js`
P-27 Service Layer: capa de servicios API (web y móvil)	Estructural	Frontend web y app móvil	`TuEventoFrontend/web/src/services/`

7. Conclusiones

El código de TuEvento aplica de forma consistente una arquitectura hexagonal con DDD sobre un monolito modular, lo que mantiene el dominio aislado de la infraestructura y permite probar los casos de uso con mocks. Los patrones de comportamiento (State, Observer, Strategy y Chain of Responsibility) concentran las reglas de negocio más sensibles: transiciones de estado de eventos y pagos, notificaciones desacopladas y moderación de contenido.

Los adaptadores hacia servicios externos (pasarela de pago, moderación de imágenes y webhooks firmados con reintentos) protegen al dominio de los cambios de proveedor, y el pipeline de Jenkins junto con el HPA de Kubernetes cubren la entrega continua y la elasticidad ante picos de venta de entradas. En los clientes, el uso de Context/Provider, hooks personalizados, una capa de servicios y un cliente HTTP con refresco de token compartido mantiene la lógica de presentación separada del acceso a datos.

Como líneas de evolución se identifican la incorporación de un proveedor de pago real mediante un nuevo adaptador de PaymentGatewayPort y el uso de un broker de mensajes si el módulo de notificaciones se extrae como servicio independiente.